

MANUAL 0.0.0.1

It's a shame that players need to read through a boring manual to play this game. However, for now, a manual seems necessary. An in-game tutorial may be added later.

Installation

Base game installation: AI training is not available, but you can use a trained AI.

1. Unzip the zipped file "UntitledGame"
2. Double-click the test.exe to run the game.

If you would like to train an AI:

1. Locate the rl_environment folder from the root folder of the game.
2. Use the "install_pinned.bat" to install the CPU training environment (RECOMMENDED). **OR**, use the "install_gpu_pinned.bat" to install the GPU training environment if you have a supported GPU (please see below).

*This game supports both CPU-only and combined CPU–GPU training. Adding a GPU will, in principle, speed up training if the CPU is fully utilised.

*The AI training environment is a Python environment.

Supporting GPUs:

Apologies to AMD users: unfortunately, because PyTorch, the framework behind the neural network training, does not support AMD GPUs on Windows, **this game cannot support AMD GPUs**. However, you can still use the CPU version, which supports both Intel and AMD CPUs.

For NVIDIA users, most **GPUs from the RTX 20XX series onwards should be supported**. During installation, a warning will appear if your GPU is not supported.

If you would like to check it manually, please follow the instructions below:

1. Visit <https://developer.nvidia.com/cuda/gpus> or <https://developer.nvidia.com/cuda/gpus/legacy> to find the Compute Capability of your GPU. If your GPU is already listed in the legacy list, it is likely unsupported.

Compute Capability	Data Center	Workstation/Consumer	Jetson
12.1		NVIDIA GB10 (DGX Spark)	
12.0	NVIDIA RTX PRO 6000 Blackwell Server Edition NVIDIA RTX PRO 4500 Blackwell Server Edition	NVIDIA RTX PRO 6000 Blackwell Workstation Edition NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition NVIDIA RTX PRO 5000 Blackwell NVIDIA RTX PRO 4500 Blackwell NVIDIA RTX PRO 4000 Blackwell NVIDIA RTX PRO 4000 Blackwell SFF Edition NVIDIA RTX PRO 2000 Blackwell GeForce RTX 5090 GeForce RTX 5080 GeForce RTX 5070 Ti GeForce RTX 5070 GeForce RTX 5060 Ti GeForce RTX 5060 GeForce RTX 5050	

- This game supports Compute Capabilities of **Turing(7.5), Ampere(8.0, 8.6), Ada(8.9), Hopper(9.0), Blackwell(10.0, 12.0).**

Fundamental concepts

Design game characters

A game character is composed of one main body and several individual “actions”. The character’s main body determines its fundamental parameters, such as size, HP, whether it flies or walks, and movement speed. The actions are literally what the character can do. For example, a character equipped with a “melee attack” action can attack nearby enemies.



A character can be assigned an unlimited number of actions, but an AI model can only handle up to five actions (see later), excluding movement and idle. Each character must have an idle action.

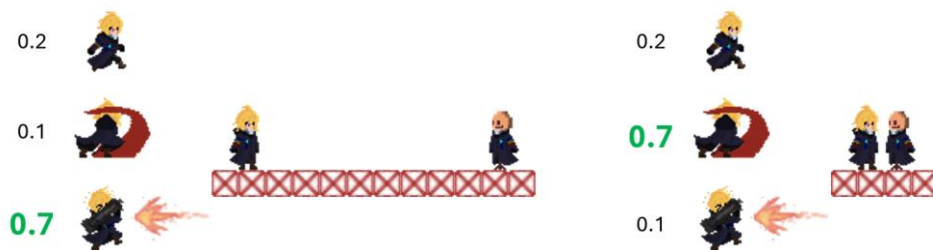
Which Action to use

In this game, one character chooses its action via three options: code, human, and AI.

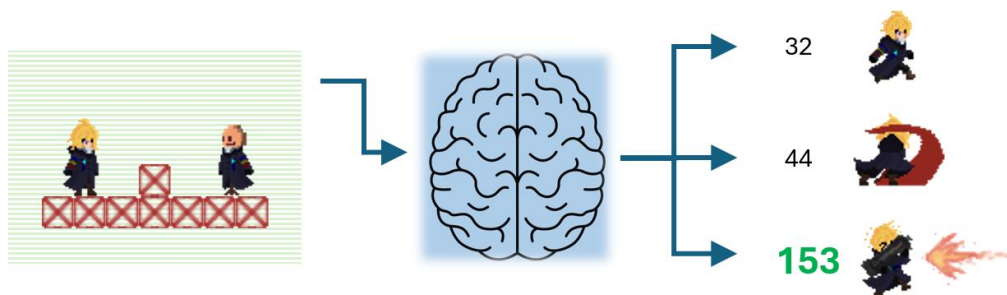
- human input** is straightforward: the user presses WASD and the space bar to move and jump, and uses LMB, RMB, and the 1–9 keys to select an action.



- **Code control** is based on hard-coded (predesigned) control logic, where the character scores all possible actions based on several factors. For example, a character whose opponent is far away will prefer a ranged attack, while one whose opponent is nearby will prefer a melee attack. When you assign a new action to a character, its code control will be added automatically by default. You can adjust key parameters of the control logic on the action page.



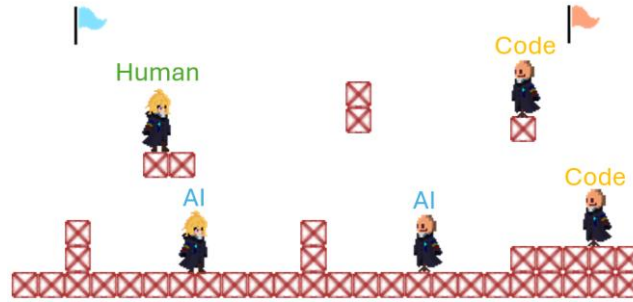
- **AI control** is based on a neural network, which needs to be trained first. The character continuously sends its observations of the surrounding environment to the neural network, which selects the best action based on those observations.



Building a playground

Just like in other auto-battler games, pick a map and place your game characters in the playground to fight. There can be up to 10 teams.

Each game character can be independently controlled by code, AI, or a human player.

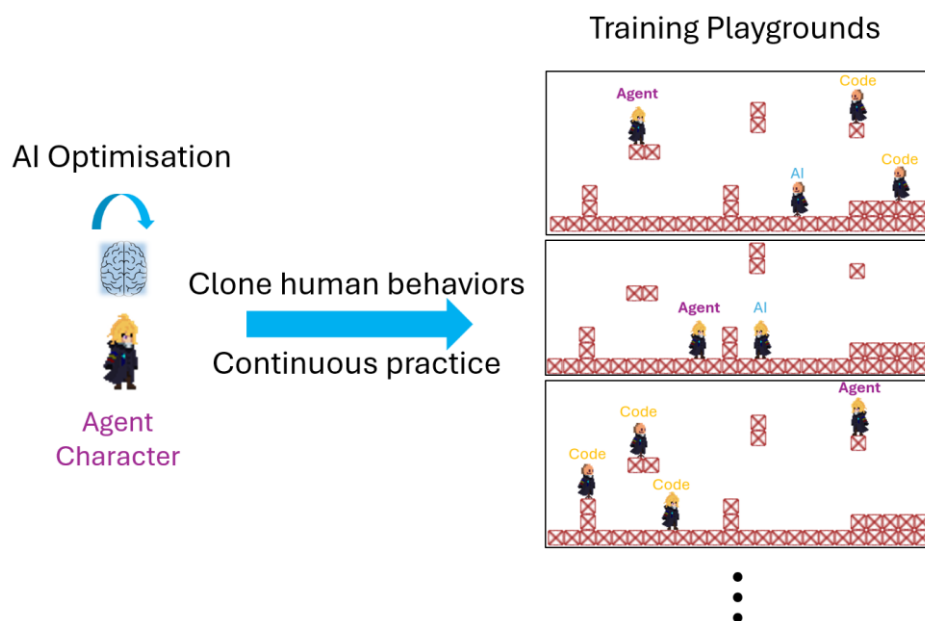


Training AI for a character

Intuitively, training an AI means letting a character “agent” controlled by the AI fight continuously in many pre-designed playgrounds, trying different strategies. The AI model then updates itself to achieve the best outcome. Or, even more straightforward, you can show the AI how to behave and let it learn. These are two training algorithms currently adopted by this game: the **Behaviour Cloning (BC)** and the **Reinforcement Learning (RL)**.

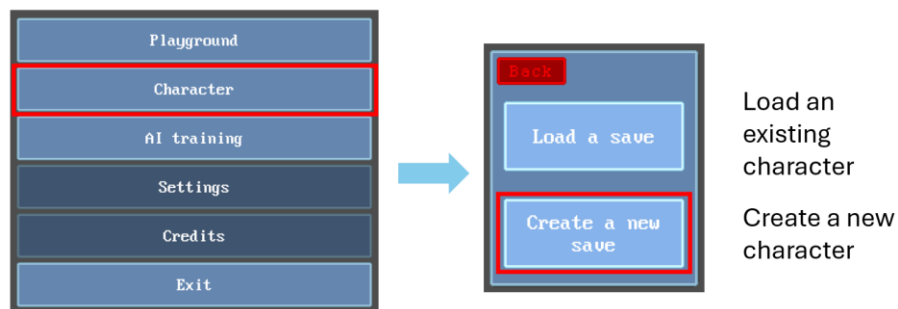
The AI model used in this game is small enough to be trained on an office laptop: I tested it on a rubbish laptop provided for free by my university, and it worked fine. With a well-designed training setup, significant progress can be observed within an hour. A high-end computer or a GPU **MAY** speed up the training process.

Please check the manual for more information.



Character

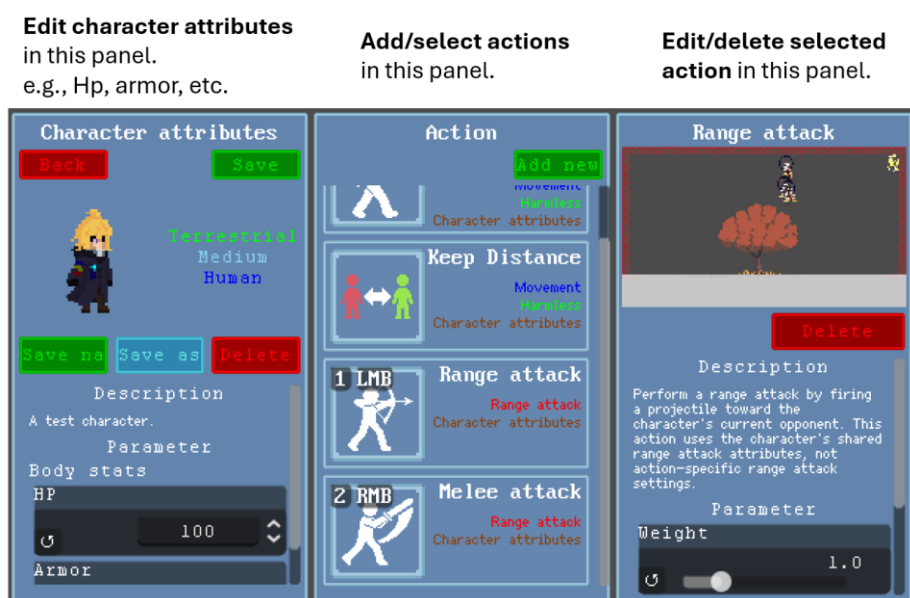
1. Go to the character creator.



2. Choose a main body for the character.



3. Customise the character.



*Hover the mouse cursor over the buttons/title for detailed explanations.

*Character save path:

C:\Users\<Windows

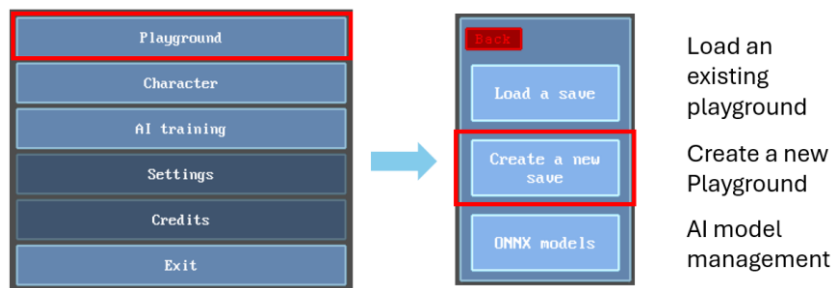
username>\AppData\Roaming\Godot\app_userdata\TestFlat\MyGame\saves\character_setups

Or, paste this into the file

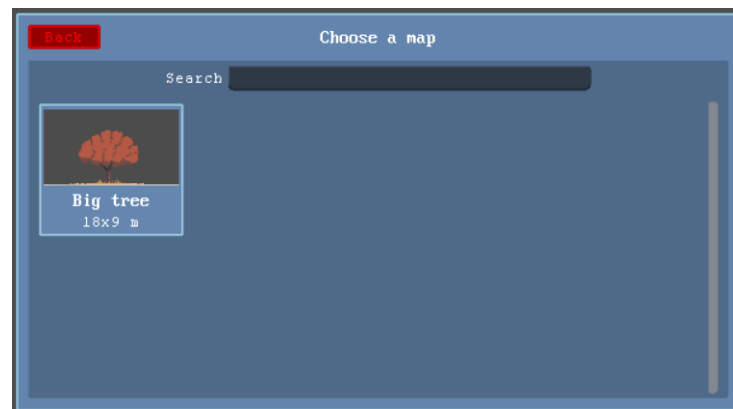
explorer: %APPDATA%\Godot\app_userdata\TestFlat\MyGame\saves\character_setups

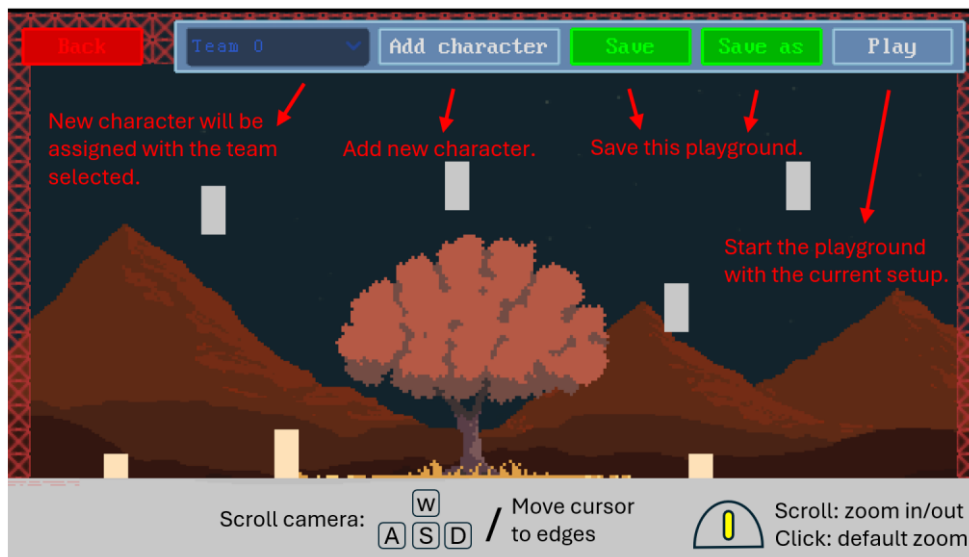
Playground

1. Go to the playground creator.



2. Choose a map.





3. Add characters.

- Character in playground.

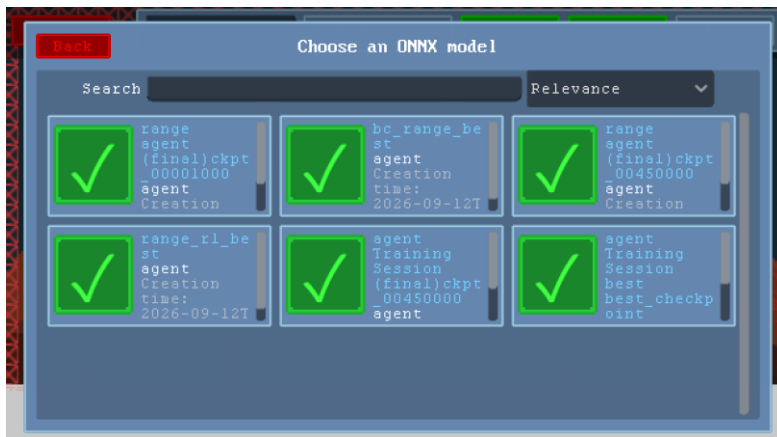


- Default control method is the code control.
- Only one character in the playground can be assigned as human player controlled.
- Key bindings for human control can be found in the action badge.



Movement uses the conventional WASD & space key bindings. Customisable key binding is currently in the Todo list.

- Assign a character with a trained AI model (ONNX format, see next section).



[Short explanation of model compatibility:

“✓”: the model fits the character.

“!”: The model was trained for this character, but some character settings have been modified since training. One important note: changing the action list, especially deleting or reordering actions, will immediately cause the model to perform poorly.

“!!”: the model was not trained for the character.

“x”: The model cannot be used for this character. This is usually because flying and walking characters use different movement systems.

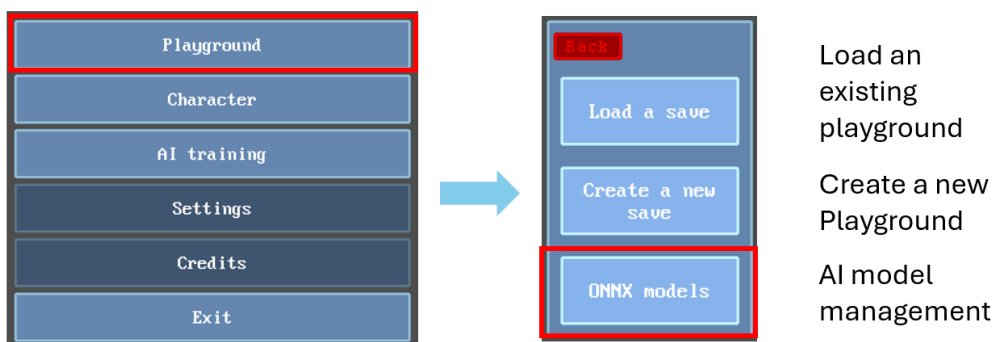
*Playground save path: C:\Users\<Windows

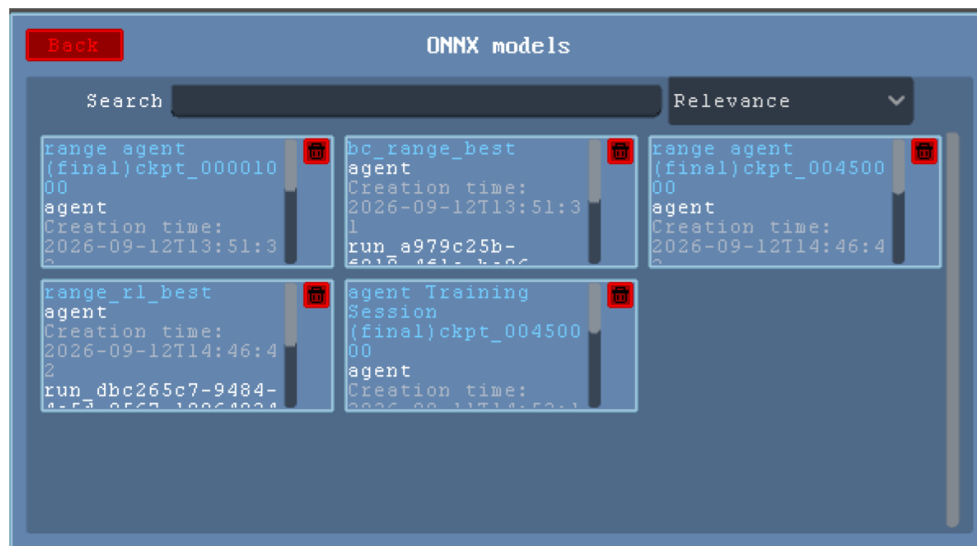
username>\AppData\Roaming\Godot\app_userdata\TestFlat\MyGame\saves\playground

Or, paste this into the file explorer: %APPDATA%\Godot\app_userdata\TestFlat\MyGame\saves\playground

AI Model Management

You can view all current AI models in the AI model manager. ONNX (Open Neural Network Exchange) is a format for AI models that allows them to run without the AI training environment.





ONNX model save path:

C:\Users\cchx7461\AppData\Roaming\Godot\app_userdata\TestFlat\MyGame\rl_model_saves\onnx

Or, paste this into the file

explorer: %APPDATA%\Godot\app_userdata\TestFlat\MyGame\rl_model_saves\onnx

AI Training

In this game, an AI model can be trained for a character “agent*” to control its actions. The “AI” in this game, in principle, is the same as the AI you may have heard a lot in recent years. However, compared with **LARGE** (language) AI models like ChatGPT, the AI model used in this game is **tiny**, so tiny that it can be trained and run on your office laptop locally (hopefully).

*We will use the term “agent” to represent the character that the AI model is trained for.

In this game, two learning methods (algorithms) can be used to train an AI model:

Behaviour cloning (BC): you show how to play, and the agent learns from recordings.

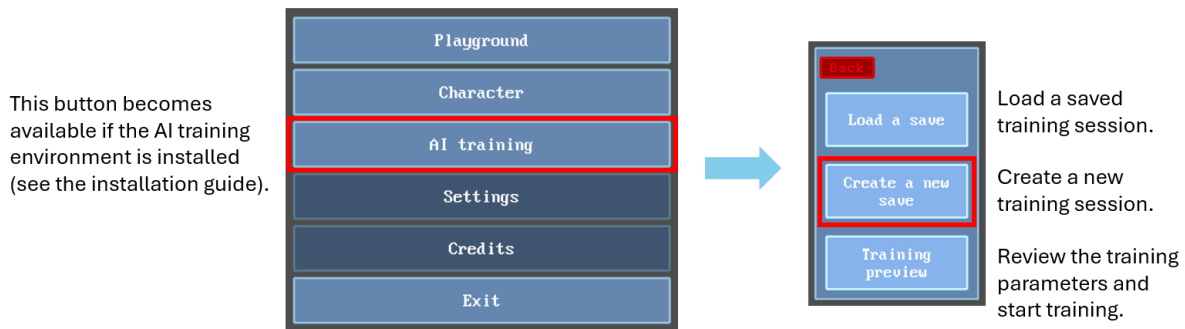
Reinforcement learning (RL): the agent improves itself by continuously trying different strategies in predesigned playgrounds. Your job is to design these training playgrounds and assign scores to different outcomes.

“**Mix and match**” is possible: an AI model trained using BC can be further trained using RL, and vice versa. In my experience, using BC to teach the agent a basic playstyle and then using RL to optimise it is a reasonable approach.

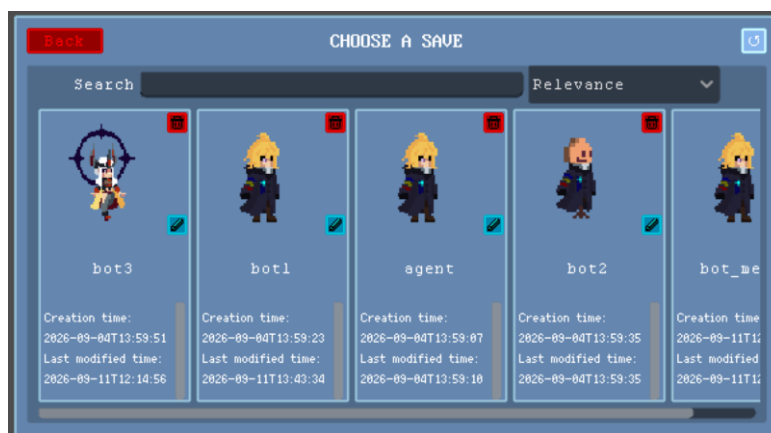
Training session

Intuitively, an AI training session is like a saved project that defines everything needed to train a neural network-based AI model for a specific agent.

1. Create an AI training session.



2. Pick the agent character from your saved characters. Note that the agent will be fixed for this training session and cannot be changed once the session is created.



3. Define AI actions. An AI model can only use actions assigned as AI actions. Pure movement actions cannot be assigned as AI actions because movement is controlled separately by the AI model.
When an action is assigned as an AI action, its logic may be simplified. For example, with the “ranged attack” action, the character will normally try to find and move to a clear shooting position before shooting. However, if it is assigned as an AI action, the character will shoot directly. Deciding when and where to shoot is the AI model’s job.
[Note that separate movement control and simplified action logic also apply to human player control.]

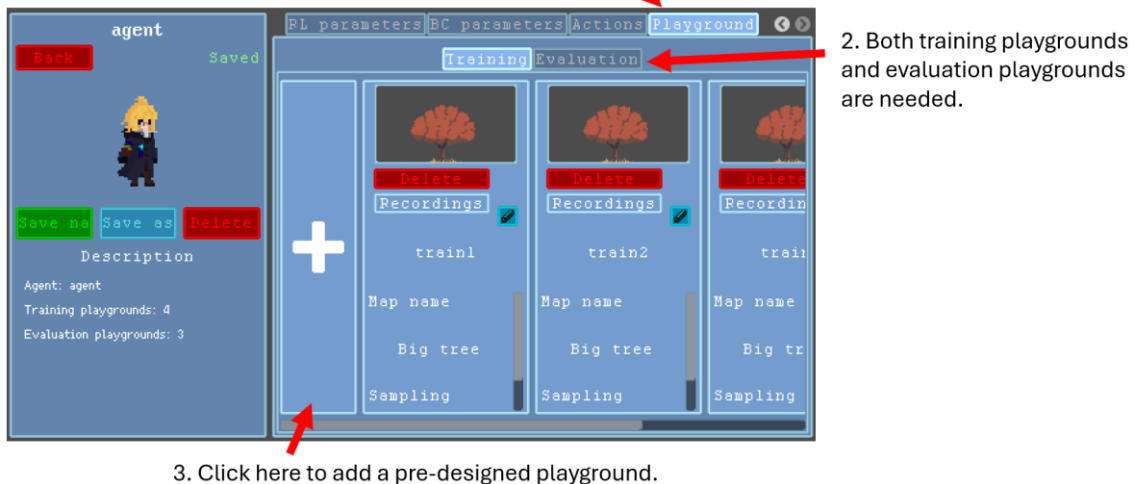
1. Find the Actions tab.



4. Define playgrounds. Select a saved playground and place the agent in it. You can change the agent's team and position, but you cannot modify the original playground.

[Training requires both training playgrounds and evaluation playgrounds. Intuitively, training playgrounds are like exercise sheets, where the agent can improve its skills through repeated practice. Evaluation playgrounds are like exams, used to assess the results. The agent does not learn from them.]

1. Find the Playground tab.



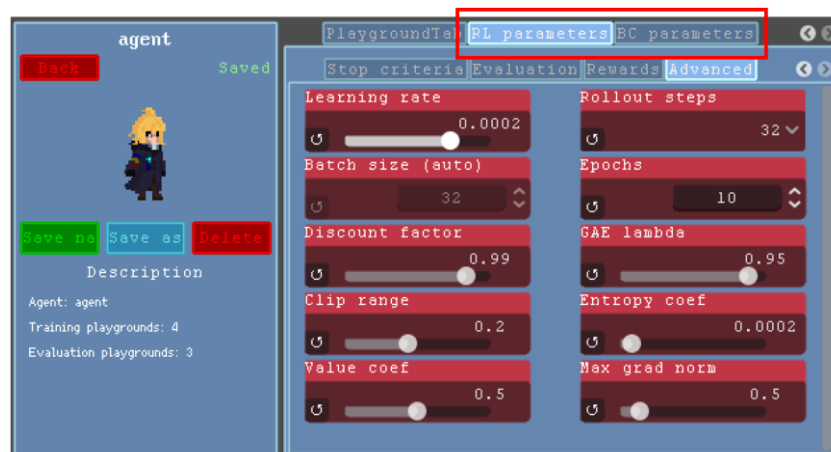


Training preparation

Before training, we still need to make specific preparations for each algorithm.

[This game provides a reasonably comprehensive set of adjustable parameters for both algorithms. You can hover your mouse cursor over each parameter's title to read its definition, but some are still quite involved. In this tutorial, I will focus on the most intuitive settings that a beginner can easily grasp.]

[If you are experienced, you can tune the other parameters without editing the Python code directly. Algorithm-related parameters can be found in the "Advanced" subtab under the RL/BC parameters tab.]

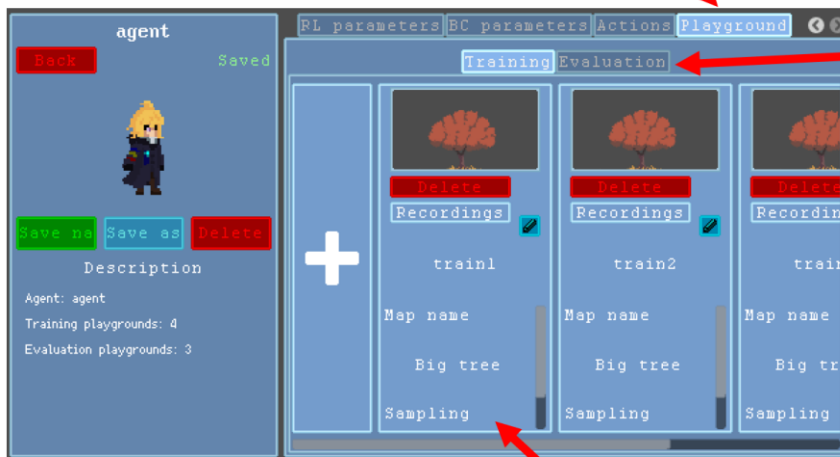


Behaviour cloning (BC) training preparation

BC literally means training the AI model to copy your behaviour. In this game, you need to record your behaviour for the AI to learn from.

- Record playthroughs.

1. Find the Playground tab.



2. Recordings are needed for both training playgrounds and evaluation playgrounds.

3. Click to load a playground.



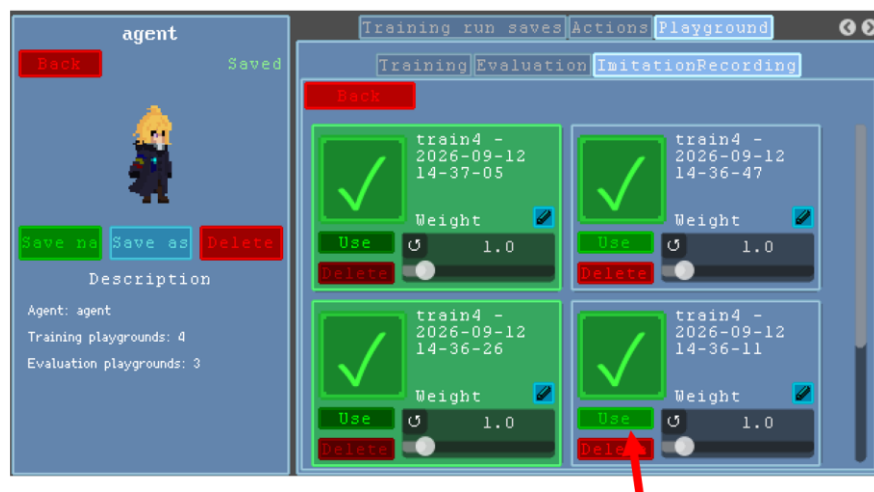
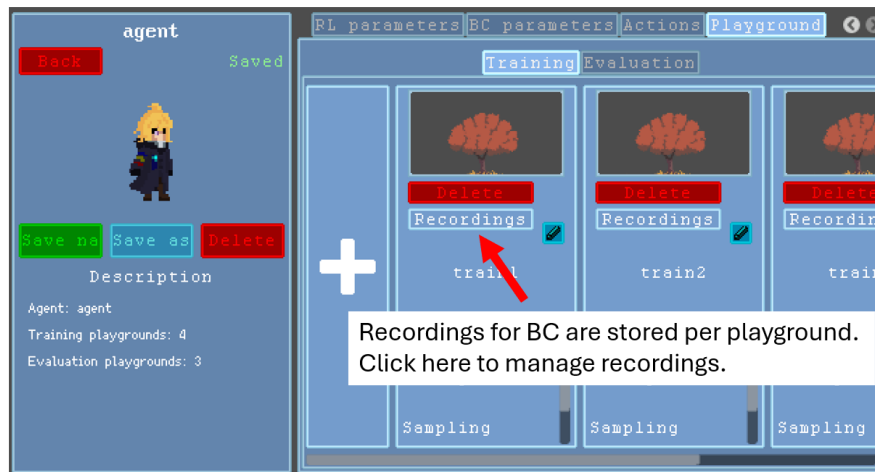
3. Click to start BC recording.

Remember that you can check the key bindings on the AI action badges.

[Recordings do not need to be perfect, even a defeated recording can be useful.]

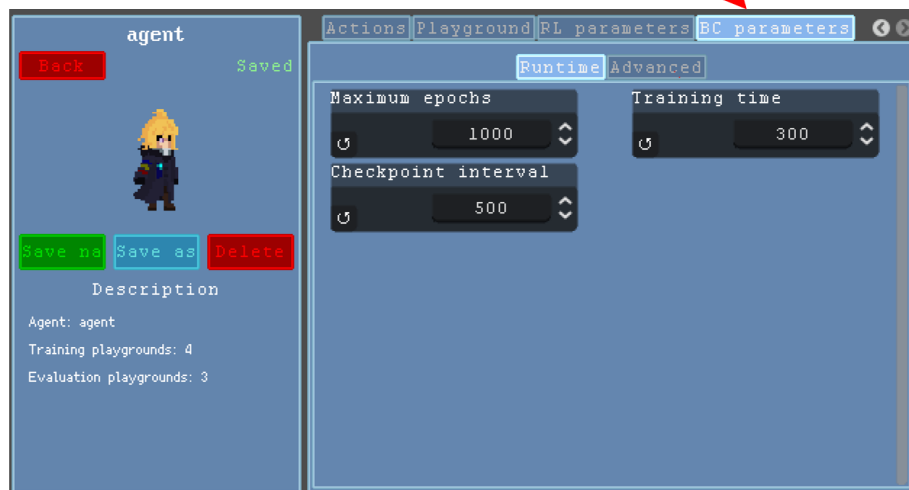
[Recordings should be prepared for both training playgrounds and evaluation playgrounds.]

- Manage recordings.



Before training, you will still need to define how long it should run. For BC, you can set a maximum number of training epochs or a time limit. Training will stop when either limit is reached. If you are unsure, simply try training with the default settings first.

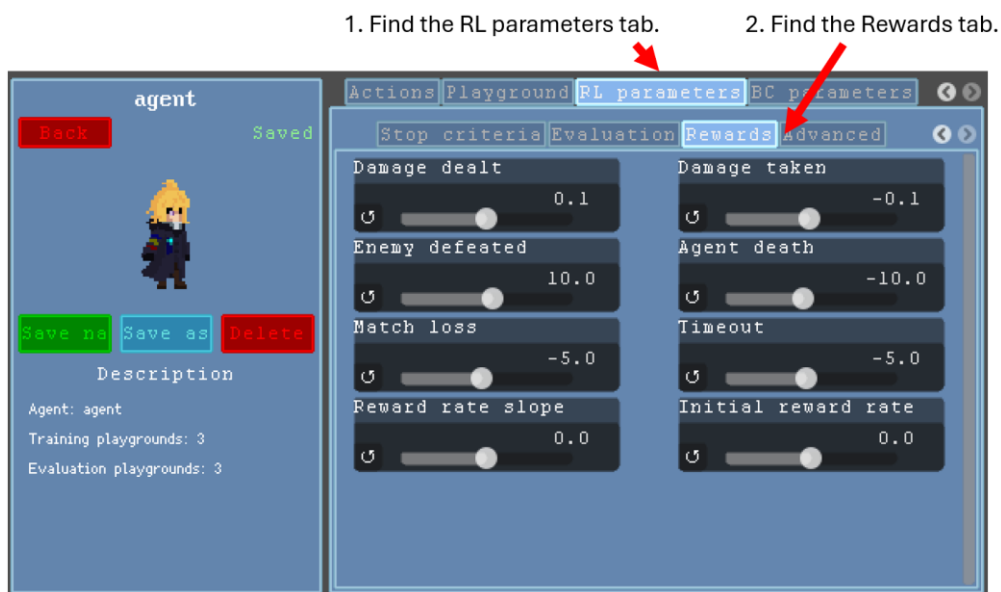
BC parameters tab.



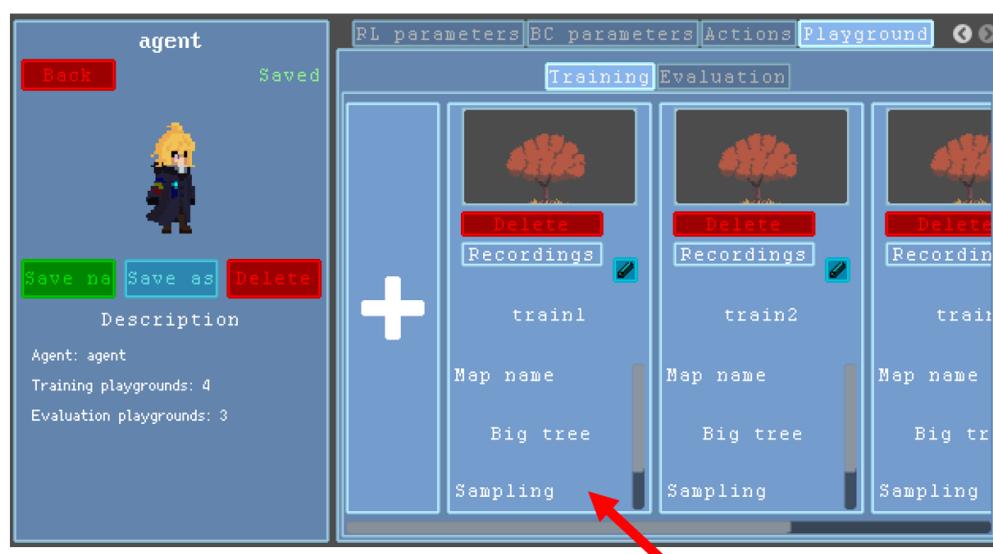
Reinforcement learning (RL) training preparation

In reinforcement learning (RL), the agent explores different actions using its current policy and learns from the rewards it receives. After adding training playgrounds, the next step is to define which events and behaviours earn rewards or incur penalties. This is like designing a marking scheme for an exam: you decide what earns or loses marks, while the agent learns a strategy that maximizes its expected cumulative reward.

- Global reward settings. These reward settings apply to all training playgrounds.



- Playground-specific settings. You should edit both training playgrounds and evaluation playgrounds.



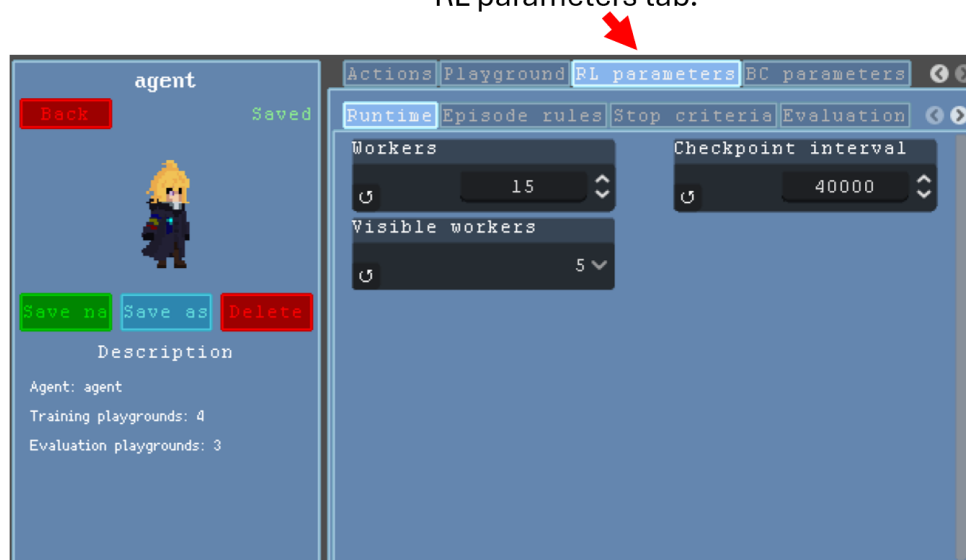
1. Click to load a playground.



These rewards are ADDED to the global reward. For example, if the “Enemy defeated” global reward is set to 10 and the “Destroy” reward for a specific character is also set to 10, destroying that character will give a total reward of 20.

- Training an AI agent using existing AI agents is possible. You can use a playground containing characters with assigned AI models as a training playground. However, due to some design limitations, these AI models will run on the CPU, increasing its load.
- Additional training parameters.

RL parameters tab.



Two important performance parameters:

1. Runtime -> Workers:

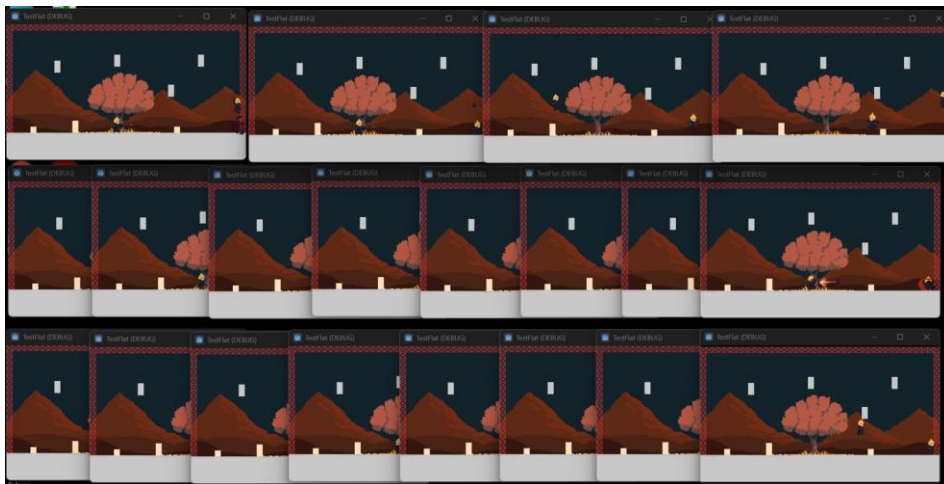
To speed up training, the game will run this many copies of itself as parallel workers. These workers are separate processes that you can view in Task Manager.

Recommendation: start with a conservative setting, such as 5 workers, to test your device's capabilities (mostly limited by CPU performance).

Setting too many workers may cause the main game to lag.

2. Runtime -> Visible workers:

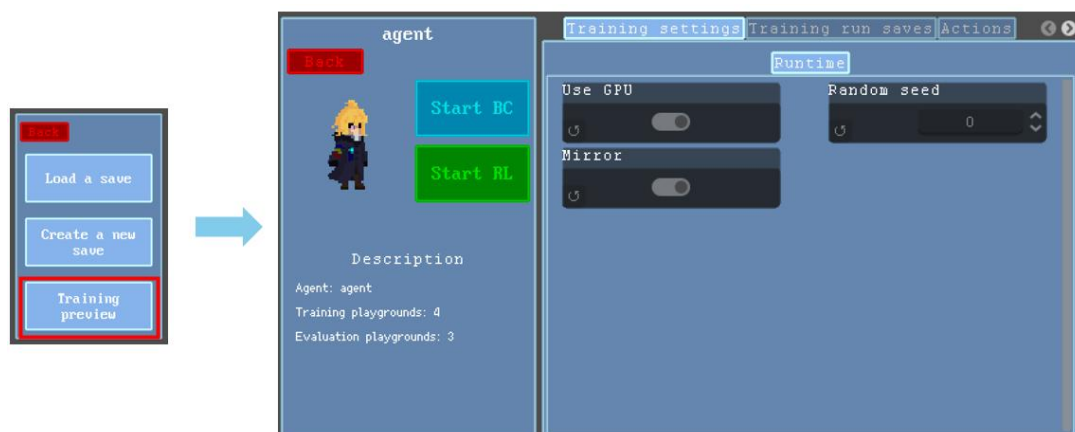
Define how many workers are displayed to you. Screenshot below: all (20) workers are visible.



RL has more parameters than BC, but if you're unsure, start with the default settings. Making parameter tuning more intuitive is one of the most challenging parts of developing this project.

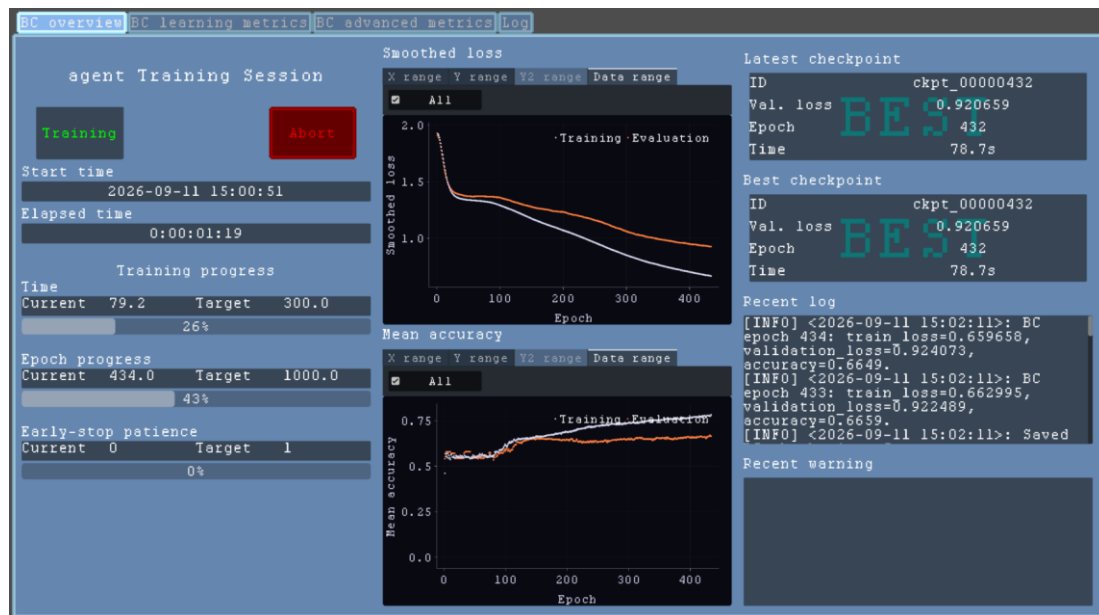
Start training from scratch

To start a training run, click the "Training preview" button and load your saved training session. Before training, you may want to review all the training settings again. Training preview mode allows you to review these values but does not allow you to modify them.

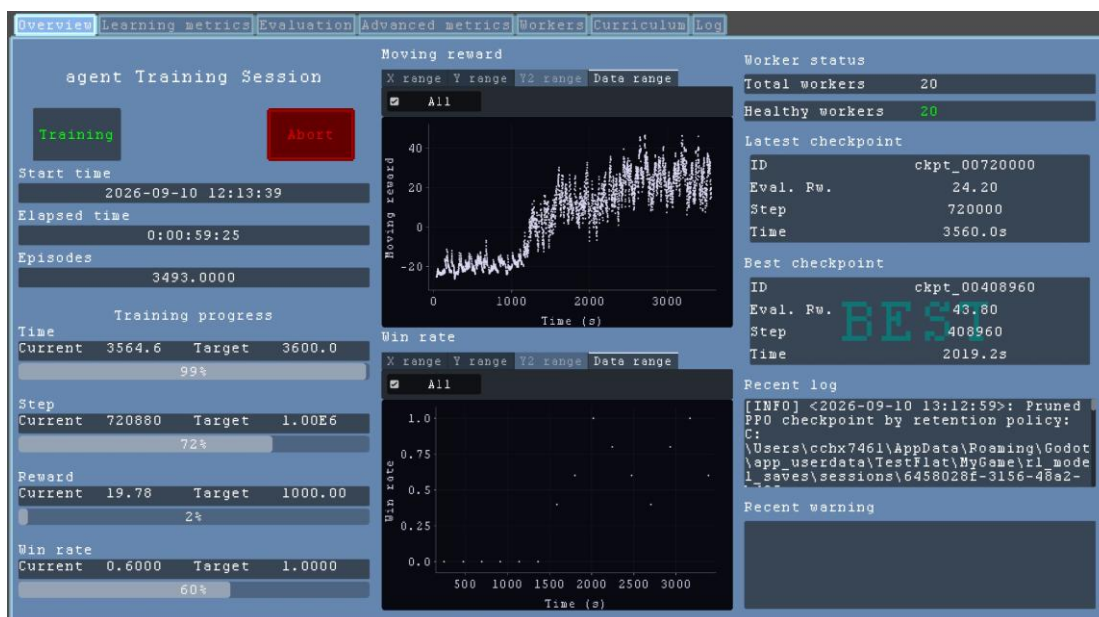


Once you are satisfied with settings, click Start BC/RL button to start training. This game provides a training monitor.

BC training monitor:



RL training monitor:



You can check the tooltips to gain a basic understanding of the different values shown in the monitor. However, in my experience, it is always more intuitive to watch the agent's actual behaviour in the displayed worker window. Sometimes, your disobedient agent will find a trick that earns a nice-looking reward without behaving the way you intended.

During training, the AI model will be saved at multiple checkpoints. The "best" checkpoint is the one where the AI performs best in the evaluation playgrounds. Note that this is not

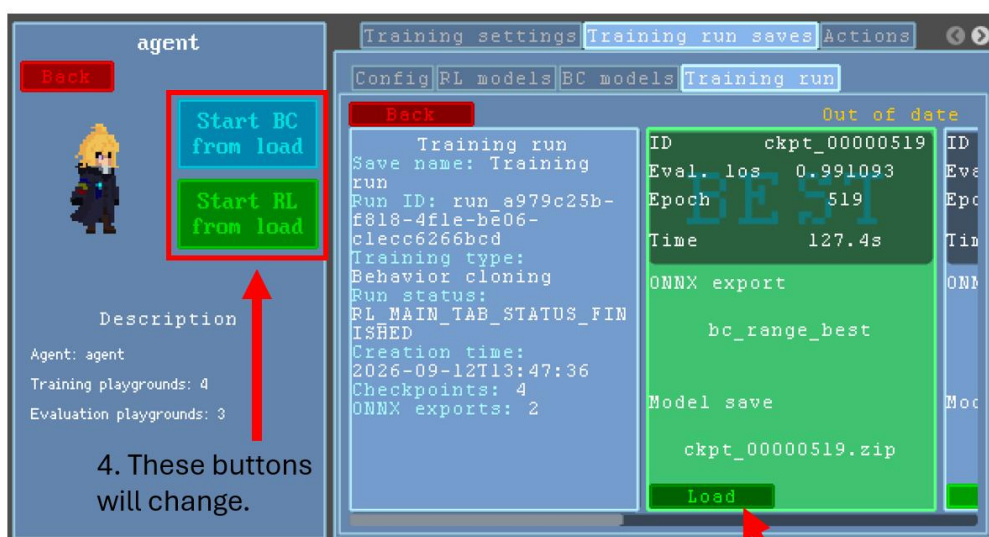
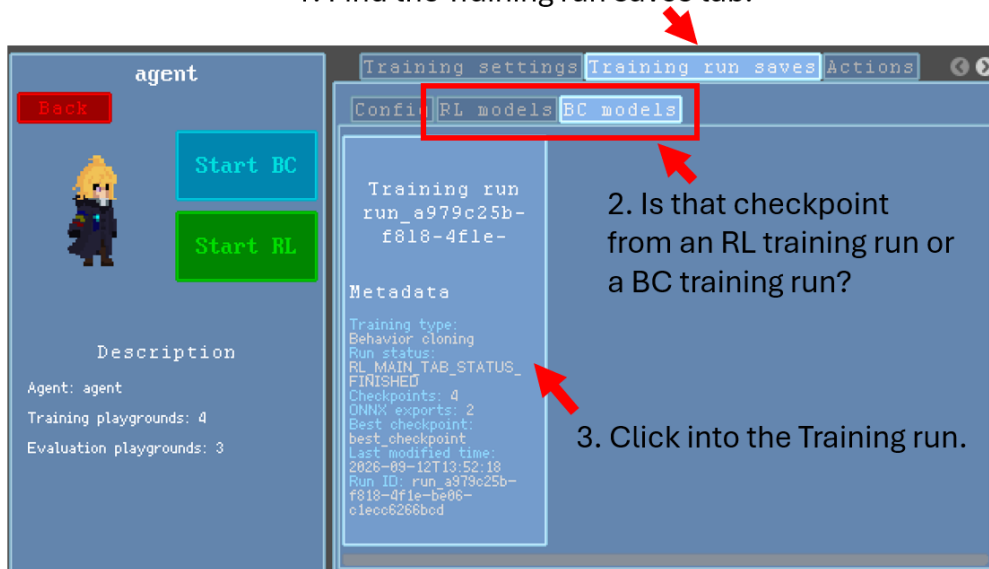
necessarily the latest checkpoint. Also, the “best” checkpoint may not truly be the best because, you know, you can get lucky in one exam without the result reflecting your true ability.

Training will stop immediately when any of the training progress bars reaches its maximum. If you do not want a particular limit to stop training, simply set it to an unreachable value. If training stops normally, the final checkpoint will be saved. However, if you manually abort training, the final checkpoint will not be saved.

Start training from a checkpoint

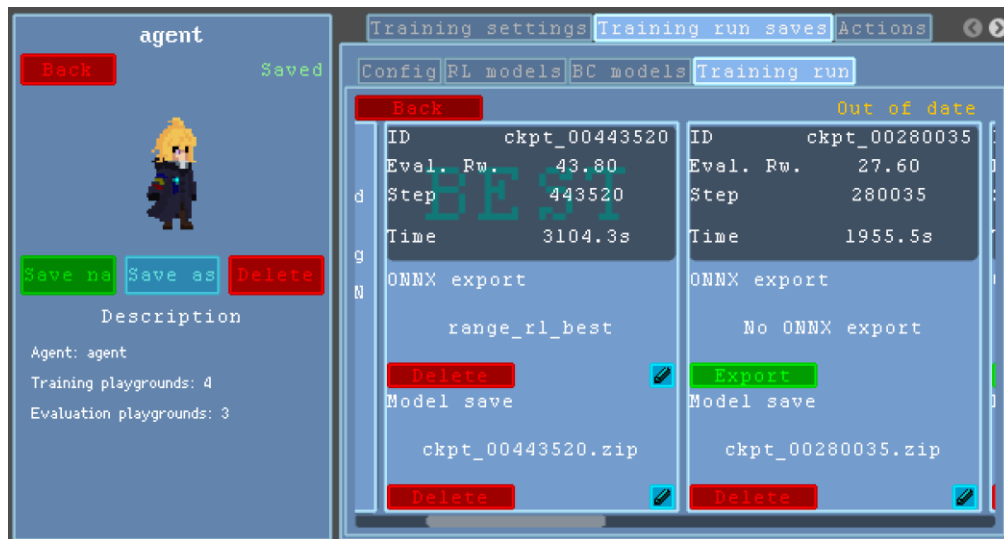
You can always train an existing model further, perhaps with different playgrounds, new actions, and so on. A model trained using BC can also be further trained using RL, and vice versa.

1. Find the Training run saves tab.



After training and ONNX export

You may access the checkpoint panel in the editing mode (not in the preview mode).



Each checkpoint has two subsections: ONNX (Open Neural Network Exchange) and Model save. Since ONNX models can run without the AI training environment installed, the current design allows you to assign an ONNX model to an agent character when designing a playground. An ONNX model can be exported from a model save, which is mainly used for training. Note that an ONNX model cannot be converted back into a model save, so be careful when deleting model saves.

ONNX models are independently saved; they can be accessed even if the training run that creates it is deleted.